

Project 1 - Bonus: Benchmarking of Refactored Code Across LLMs and Settings

(CS6.401 – Software Engineering)

TEAM 33

Members:

- Aryan Mishra, 2024121001
- Aditya Nair, 2023111029
- Varun Patil, 2025204013
- Hardik Chadha, 2023111031
- Vijay Aravynthan, 2023111007

Bonus Task: Benchmarking

We compared two LLMs (Claude 3.5 Sonnet and Gemini 1.5 Pro) in both Non-Agentic (single-shot prompt with file context) and Agentic (autonomous tool use) settings. The goal was to benchmark their ability to identify and refactor the Hub-like God Object smell in WeblogEntry.java (Issue #4), which we manually refactored by extracting classes like WeblogEntryPresenter, TagManifest, and PluginConfiguration.

Setup

- File Addressed: WeblogEntry.java
- Models Used: Claude 3.5 Sonnet (Thinking mode), Gemini 3 Pro (Preview).
- Settings: Non-Agentic (provided WeblogEntry.java and designCodeSmells.csv), Agentic (tool-based repo exploration).

1. Smell Identification & Focus

Setting	Model	Identified God Object?	Decided to Fix?	Focus Area
Manual	Human Team	Yes	Yes	Architectural Decoupling
Non-Agentic	Claude Sonnet	Yes	No	Internal Encapsulation
Non-Agentic	Gemini 3 Pro	Yes	No	Local Logic Simplification
Agentic	Claude Sonnet	Yes	Partial	Analysis Stage Only
Agentic	Gemini 3 Pro	Yes	Partial	Refactor Attempts

Observation:

In the Non-Agentive setting, both models correctly identified the high complexity or specific smells provided in the CSV. However, they refused or failed to address the Hub-like God Object smell. Claude Sonnet explicitly stated that such smells are architectural and require changes across many files. Gemini 3 Pro focused on method-level refactoring instead of class-level extraction. In the Agentive setting, despite having the ability to create new files and explore dependencies, the models still trended towards conservative, single-file improvements rather than full architectural restructuring.

2. Quality of Refactoring & Comliability

Metric	Manual Refactoring	Non-Agentive LLMs	Agentive LLMs
Architectural Change	High (3 new classes)	None	Low/None
De-coupling	High	Low	Low
Comliability	High (validated)	High	Variable

Conclusion

1. Identification vs. Action: LLMs are excellent at identifying smells but prioritize Extract Method and Encapsulation over Extract Class for God Objects. 2. Context Limitation: Non-Agentive LLMs recognized their limitations regarding cross-file architectural changes. 3. Agentive Behavior: Even with tool access, models did not spontaneously perform large-scale architectural extraction. Verdict: For architectural refactoring across files, Manual Human Engineering remains superior. AI is best suited for local refactoring within identified God Objects.